

Практическая работа по Информатике

Строки. Параметры командной строки. Оператор выбора

При обработке строк ряд встроенных процедур и функций может существенно облегчить жизнь, хотя все эти операции можно и нужно уметь выполнять и на уровне отдельных символов. Рассмотрим некоторые из таких подпрограмм.

Длина строки: функция `length` принимает на вход выражение типа `string` и возвращает целое число — длину строки. Текущую длину строки можно изменить принудительно процедурой `SetLength`; например, после выполнения

```
s:='Free Pascal Compiler';  
SetLength(s,4);
```

в переменной `s` будет содержаться строка «Free». Учтите, что `SetLength` может также и увеличить длину строки, в результате чего в конце её окажется «мусор» — непонятные символы, которые лежали в этом месте памяти до того, как там разместили строку. Учтите, что меняется только текущая длина строки, но никоим образом не размер области памяти, которая под эту строку выделена. В частности, если вы опишете строковую переменную на десять символов, а затем попытаетесь установить ей длину, превышающую 10, ничего хорошего у вас не получится: переменная `s10` не может содержать строку длиннее десяти символов:

```
s10: string[10];  
begin  
    SetLength(s,12)      {ошибка!}
```

Встроенные функции `LowerCase` и `UpCase` принимают на вход выражение типа `string` и возвращают тоже `string` — такую же строку, какая содержалась в параметре, за исключением того, что латинские буквы оказываются приведены соответственно к нижнему или к верхнему регистру (иначе говоря, первая функция в строке заменяет заглавные буквы на строчные, а вторая, наоборот, строчные на заглавные).

Функция `copy` принимает на вход три параметра: строку, начальную позицию и количество символов, и возвращает подстроку заданной строки, начиная с заданной начальной позиции, длиной как заданное количество символов, либо меньше, если символов в исходной строке не хватило. Например, `copy('abracadabra', 3, 4)` вернёт строку 'raka', а `copy('foobar', 4, 5)` — строку 'bar'.

Процедура `delete` тоже принимает на вход строку (на этот раз через параметр-переменную, потому что строка будет изменена), начальную позицию и количество символов, и удаляет из данной строки (прямо на месте, то есть в той переменной, которую вы передали параметром) заданное количество символов, начиная с заданной позиции (либо до конца строки, если символов не хватило). Например, если в переменной `s` содержалось всё то же «Free Pascal Compiler», то после выполнения `delete(s, 5, 6)` в переменной `s` окажется «Free Compiler»; а если бы мы применили `delete(s, 5, 100)`, получилось бы просто «Free».

Встроенная процедура `insert` вставляет одну строку в другую. Первый параметр задаёт вставляемую строку; вторым параметром задаётся переменная строкового типа, в которую надлежит вставить заданную строку. Наконец, третий параметр (целое число) указывает позицию, начиная с которой следует произвести вставку. Например, если в переменную `s` занести строку «abcdef», а потом выполнить `insert('PQR', s, 4)`, то после этого в переменной `s` будет находиться строка «abcPQRdef».

Функция `pos` принимает на вход две строки: первая задаёт подстроку для поиска, вторая — строку, в которой следует производить поиск. Возвращается целое число, равное позиции

подстроки в строке, если таковая найдена, или 0, если не найдена. Например, `pos('Pascal', 'Free Pascal Compiler')` вернёт 6, а `pos('aaa', 'Free Pascal Compiler')` вернёт 0.

Может оказаться очень полезной процедура `val`, которая строит число типа `longint`, `integer` или `byte` по его строковому представлению. Первым параметром в процедуру подаётся строка, в которой должно содержаться текстовое представление числа (возможно, с некоторым количеством пробелов перед ним); вторым параметром должна быть переменная типа `longint`, `integer` или `byte`; третьим параметром указывается ещё одна переменная, всегда имеющая тип `word`. Если всё прошло успешно, во второй параметр процедура занесёт полученное число, в третий — число 0; если же произошла ошибка (то есть строка не содержала корректного представления числа), то в третий параметр заносится номер позиции в строке, где перевод в число дал сбой, а второй параметр в этом случае остаётся неопределённым.

Обратную задачу — преобразование числа в строку — выполняет процедура `str`(число, строка). Например, если число вещественное, то указываем сначала общее число знаков (7), а затем число знаков после запятой (3): `str(x:7:3, s)`. Если число целое, то `str(number, s)`.

Параметры командной строки

Программу можно запускать с аргументами командной строки, например:

```
./demo one two three
```

В Паскале эти параметры доступны с помощью встроенных функций:

- `ParamCount` - возвращает количество параметров ком. строки;
- `ParamStr` — принимает на вход целое число и возвращает строку (string), соответствующую параметру командной строки с заданным номером. При этом имя программы считается параметром номер 0.

Пример программы, которая печатает все элементы своей командной строки:

```
program cmdline;
var
    i: integer;
begin
    for i := 0 to ParamCount do
        writeln('[', i, ']: ', ParamStr(i))
    end.
./cmdline one two three
[0]:./cmdline
[1]:one
[2]:two
[3]:three
```

Оператор выбора – обобщение оператора ветвления

В отличие от ветвления, где только 2 варианта исполнения, здесь можно задать любое количество вариантов. Во время выполнения программы вычисляется заданное выражение, которое может иметь любой порядковый тип; в зависимости от полученного значения выбирается одна из предусмотренных в теле оператора ветвей выполнения.

Формат оператора выбора:

```
case арифметическое выражение порядкового типа of
    значение 1:
        оператор;
```

```
значение 2:
    оператор;
...
...
значение n:
    оператор;
else
    оператор
end
```

Пример, как использовать оператор выбора:

```
case month of
    January: begin
        ...;
    end;
    February: begin
        ...;
    end;
    . . .
    December: begin
        ...;
    end;
else
    ...
end
```

В качестве значений переменной month в этом примере надо применять значения специально введённого перечислимого типа, при этом выбрав осмысленные идентификаторы, но не числа!

Операторы выбора не допускается вкладывать друг в друга. Выделяйте каждую альтернативу в подпрограмму.

Задание 1.

Напишите программу, используя встроенные подпрограммы для работы со строками. Используйте свой номер студенческого билета в качестве исходной строки.

1.1 Удалите символы из своего номера, чтобы получить номер вашей группы и выведите этот номер на экран.

1.2 Затем с помощью конкатенации снова добавьте необходимые символы к номеру группы и восстановите свой номер студ. билета, выведите его на экран.

1.3 Найдите номер группы в своём номере студ. билета и снова выведите его на экран.

1.4 Преобразуйте строку с номером своего студ. билета в число, умножьте это число на 9 и выведите результат на экран.

1.5 Преобразуйте полученное число в строку, вставьте символ точки после 4-го символа и выведите результат на экран.

Задание 2.

Напишите программу, которая выбирает и печатает по-русски буквами день недели, соответствующий введенному английскому слову. Слова можно выбрать из этого списка: Monday, Tuesday, Wednesday, Thursday, Friday, Saturday, Sunday.